SECTION A

Question1 : Justify your constraint choices for at least three columns across two tables. What data integrity problems would arise if these constraints were not applied?

Answer:

Constraints are used to maintain data accuracy and integrity in a database.

1. **PRIMARY KEY:** The restaurant_id column in the Restaurants table is a primary key because every restaurant must have a unique ID. Without it, duplicate or NULL IDs could make records difficult to identify.
2. **NOT NULL:** The name column in the Restaurants table should have a NOT NULL constraint because every restaurant must have a name. Without it, restaurant records could be incomplete.
3. **FOREIGN KEY:** The restaurant_id column in the Orders table should be a foreign key referencing the Restaurants table. Without it, orders could refer to restaurants that do not exist, creating invalid relationships.

Thus, these constraints help maintain data integrity and consistency.

Question2 : Explain why the WHERE clause is essential in each DML statement. What could go wrong with an UPDATE or DELETE executed without a proper WHERE condition

Answer:

The WHERE clause is essential in DML statements because it specifies which records should be updated or deleted.

In an UPDATE statement, WHERE ensures that only the intended records are modified. For example, it can update orders that are Pending and older than seven days to Cancelled.

In a DELETE statement, WHERE ensures that only the required records are removed, such as unavailable menu items from closed restaurants.

Without a proper WHERE condition, an UPDATE may modify all rows, and a DELETE may remove all rows from the table. This can cause data loss and incorrect information.

Question 3 : Which SQL clauses and aggregate functions would you use to produce this report? Explain why HAVING must be used here rather than WHERE to filter by aggregate results.

Answer:

The SUM(), AVG(), and COUNT() aggregate functions are used to calculate total revenue, average order value, and total number of orders, respectively. The WHERE clause filters orders from the current month, while GROUP BY groups the orders by restaurant.

The HAVING clause is used to display only restaurants that have processed more than 20 orders. HAVING is necessary because it filters grouped results using aggregate functions, whereas WHERE filters individual rows before grouping.

Question 4 : Compare INNER JOIN and LEFT JOIN for this query. Which would you use and why? Describe what each join type does to records that have no match in the other table

Answer:

INNER JOIN returns only the records that have matching values in both tables. Orders referring to restaurants that no longer exist will be excluded from the result.

LEFT JOIN returns all records from the left table (orders) and matching records from the restaurants table. If no matching restaurant exists, the restaurant name will be NULL.

Therefore, I would use LEFT JOIN because the requirement is to display every order, including orders whose restaurant is no longer in the table.

Question 5 : Justify why a VIEW is preferred over direct table access here. What are two limitations the front-end developer should be aware of when working with this VIEW?

Answer:

A VIEW named top_restaurants is preferred over direct table access because it provides a simplified interface to the front-end team. It displays only the required information, such as restaurant name, total orders, and average rating, while hiding unnecessary ID columns. It also improves security and reduces the need to write complex queries repeatedly.

Two limitations:

1. A VIEW containing aggregate functions and GROUP BY is generally not directly updatable.
2. Complex VIEWs involving multiple joins and aggregate calculations may affect query performance.

Question 6 : Which database mechanism should have prevented this inconsistency? Explain how COMMIT, ROLLBACK, and SAVEPOINT work together to ensure both operations succeed or neither takes effect

Answer :

To prevent data inconsistency in the payment module, a **transaction** should be used to ensure that wallet deduction and payment insertion are completed together. If any operation fails, the transaction can be rolled back.

- **COMMIT:** Permanently saves all changes made during a successful transaction.
- **ROLLBACK:** Undoes uncommitted changes when an error occurs, helping maintain data consistency.
- **SAVEPOINT:** Creates a checkpoint within a transaction to which we can roll back without undoing the entire transaction.

Thus, transactions help ensure that the wallet deduction and payment record remain consistent.